Unit 2: Linear Algebra and Multivariable Calculus

Charles Rambo

UCLA Anderson

2026

Table of Contents I

1. Linear Algebra
   - Vectors and Vector Spaces
   - Matrices
   - Linear Transformations
   - Coordinate and Matrix Representation
   - Inner Product Spaces
   - Norms and Distances
   - Projections
   - Determinants
   - Cramer's Rule

Table of Contents II

- Eigenvectors and Eigenvalues
- Positive Definite Matrices

2. Multivariable Calculus
   - Partial Derivatives
   - Gradient Vectors
   - Maximum and Minimum Values
   - Multiple Integrals
   - Change of Variables

Linear Algebra

Vector Definition

Definition

- A **vector** is a quantity that has both direction and magnitude.
- For this class, a **scalar** is simply an element of $\mathbb{R}$.

In introductory texts, a vector is usually written with a bold letter or an arrow over the letter, e.g. $\boldsymbol{v}$ or $\vec{v}$, and no special notation is used for a scalar. However, beyond introductory texts, typically no special notation is used for a vector either. Whether a quantity is a vector or scalar is implied by the context. We will *mostly* follow the convention of introductory texts here.

Vector Space Definition

Definition

A **real vector space** V is a set such that, for $\boldsymbol{u}$, $\boldsymbol{v}$, and $\boldsymbol{w}$ in V and α and β in $\mathbb{R}$, the following hold.

VS1 $(\boldsymbol{u}+\boldsymbol{v})+\boldsymbol{w}=\boldsymbol{u}+(\boldsymbol{v}+\boldsymbol{w})$

VS2 There is an element $\mathbf{0}$ in V such that $\mathbf{0}+\boldsymbol{u}=\boldsymbol{u}+\mathbf{0}=\boldsymbol{u}$

VS3 There exists $-\boldsymbol{u}$ in V such that $\boldsymbol{u}+(-\boldsymbol{u})=(-\boldsymbol{u})+\boldsymbol{u}=\mathbf{0}$

VS4 $\boldsymbol{u}+\boldsymbol{v}=\boldsymbol{v}+\boldsymbol{u}$

VS5 $\alpha(\boldsymbol{u}+\boldsymbol{v})=\alpha\boldsymbol{u}+\alpha\boldsymbol{v}$

VS6 $(\alpha+\beta)\boldsymbol{u}=\alpha\boldsymbol{u}+\beta\boldsymbol{u}$

VS7 $(\alpha\beta)\boldsymbol{u}=\alpha(\beta\boldsymbol{u})$

VS8 $1\boldsymbol{u}=\boldsymbol{u}$

Subspace Definition

Definition

Let V be a vector space and let W be a subset of V. Then W is a **subspace** of V if the following properties hold.

(i) $\boldsymbol{w}_1$ and $\boldsymbol{w}_2$ in W implies $\boldsymbol{w}_1 + \boldsymbol{w}_2$ is in W

(ii) α in $\mathbb{R}$ and $\boldsymbol{w}$ in W implies $\alpha\boldsymbol{w}$ is in W

(iii) The element $\mathbf{0}$ is in W

Vector Space and Subspace Example

Example

The set $V = \{f : \mathbb{R} \to \mathbb{R} | f \text{ continuous}\}$ is a real vector space and $W = \{f : \mathbb{R} \to \mathbb{R} | f \text{ differentiable}\}$ is a subspace, if for f and g in V, we define $f + g$ to be the function which satisfies

$$(f + g)(x) = f(x) + g(x)$$

and for α in $\mathbb{R}$ we define αf to be the function which satisfies

$$(\alpha f)(x) = \alpha f(x).$$

Linear Combination Definition

Definition

A **linear combination** of $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ is

$$\alpha_1 \boldsymbol{v}_1 + \alpha_2 \boldsymbol{v}_2 + \ldots + \alpha_n \boldsymbol{v}_n,$$

where α_i is an element of $\mathbb{R}$.

Linear Independence Definition

Definition

Let $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ be a set of elements of V. Then the set is **linearly independent** if for $\alpha_1, \alpha_2, \ldots, \alpha_n$ in $\mathbb{R}$,

$$\alpha_1 \boldsymbol{v}_1 + \alpha_2 \boldsymbol{v}_2 + \ldots + \alpha_n \boldsymbol{v}_n = \mathbf{0} \qquad \text{implies} \qquad \alpha_i = 0 \text{ for all } i.$$

If $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ is not linearly independent, then it is **linearly dependent**.

Span Definition

Definition

The **span** of $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_m\}$ is

$$\text{span}\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_m\} = \{\alpha_1 \boldsymbol{v}_1 + \alpha_2 \boldsymbol{v}_2 + \ldots + \alpha_m \boldsymbol{v}_m | \alpha_i \in \mathbb{R}\}.$$

Basis Definition

Definition

The set $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ is a **basis** of V if

(i) $\text{span}\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\} = V$, and

(ii) the set $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ is linearly independent.

The Standard Basis

Consider $\mathbb{R}^n$ as a real vector space. If we define

$$\begin{aligned} \boldsymbol{e}_1 &= (1, 0, \ldots, 0, 0) \\ \boldsymbol{e}_2 &= (0, 1, \ldots, 0, 0) \\ &\vdots \\ \boldsymbol{e}_{n-1} &= (0, 0, \ldots, 1, 0) \\ \boldsymbol{e}_n &= (0, 0, \ldots, 0, 1), \end{aligned}$$

then $\{\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_{n-1}, \boldsymbol{e}_n\}$ is a basis for $\mathbb{R}^n$. This basis is called the **standard basis** of $\mathbb{R}^n$.

Basis Elements and Independence

Theorem

Let V be a vector space, and suppose $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_m\}$ is a basis of V. If $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots \boldsymbol{w}_n$ are elements of V and $n > m$, then $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_n$ are linearly dependent.

If we had two bases with n and m elements, we could swap their roles. If $n > m$, the n elements are dependent (contradicting the definition of a basis). If $m > n$, the same contradiction occurs. Thus, n must equal m. In other words, the number of elements in a basis of a vector space is unique.

Dimension Definition

Since the number of elements in a basis is unique, this means that the quantity is well-defined for any finite dimensional vector space. As a result, we give this unique quantity a name.

Definition

The **dimension** of a vector space V is the number of elements in any basis of V.

Matrix Arithmetic

Example

Suppose

$$A = \begin{pmatrix} 1 & 2 & 3 \\ -1 & 0 & 2 \end{pmatrix} \qquad \text{and} \qquad B = \begin{pmatrix} -1 & 5 & -2 \\ 2 & 2 & -1 \end{pmatrix}.$$

Compute (a) $2A + B$ and (b) AB^T.

Python Matrix Arithmetic

Example

Suppose

$$C = \begin{pmatrix} 2 & 1 & 3 & 4 \\ -3 & 1 & 5 & 1 \\ 5 & -1 & 11 & 7 \\ -1 & 10 & 2 & 4 \end{pmatrix} \quad \text{and} \quad D = \begin{pmatrix} 100 & -5 & 2 & 1 \\ 7 & -2 & 1 & 1 \\ -5 & 1 & 2 & 3 \\ 20 & 1 & 4 & 50 \end{pmatrix}.$$

Use Python to compute (a) $2C + D$ and (b) CD^T.

Python Matrix Arithmetic Solution

```python
# Import module
import numpy as np

# Define matrices
C = np.array([[2, 1, 3, 4],
              [-3, 1, 5, 1],
              [5, -1, 11, 7],
              [-1, 10, 2, 4]])

D = np.array([[100, -5, 2, 1],
              [7, -2, 1, 1],
              [-5, 1, 2, 3],
              [20, 1, 4, 50]])

# Perform arithmetic
result_a = 2 * C + D
result_b = C @ D.T
```

Python Matrix Arithmetic Result

The results are

$$\texttt{result_a} = \begin{pmatrix} 104 & -3 & 8 & 9 \\ 1 & 0 & 11 & 3 \\ 5 & -1 & 24 & 17 \\ 18 & 21 & 8 & 58 \end{pmatrix}$$

and

$$\texttt{result_b} = \begin{pmatrix} 205 & 19 & 9 & 253 \\ -294 & -17 & 29 & 11 \\ 534 & 55 & 17 & 493 \\ -142 & -21 & 31 & 198 \end{pmatrix}.$$

`np.matmul` and `@`

The operator `@` was introduced in Python 3.5, and is equivalent to `np.matmul`. See the NumPy documentation for more details.

The Identity Matrix

The $n \times n$ identity matrix I is such that $AI = A$ for A an $m \times n$ matrix and $IB = B$ for B an $n \times m$ matrix. This matrix has ones on the main diagonal and zeros elsewhere. For example, the 3×3 identity matrix is

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

In Python, the command for the $n \times n$ identity matrix is `np.eye(n)`.

Inverse of a Matrix

If A is an $n \times n$ **invertible** or **non-singular** matrix, there is an $n \times n$ matrix B such that $AB = BA = I$. We typically call B the **inverse** of A and write it as A^{-1}. In Python, the command is `np.linalg.inv(A)`.

Useful Inverse Matrix Formula

Suppose

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

Then

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

whenever the formula makes sense.

Inverse Example

Example

Compute the inverse of

$$A = \begin{pmatrix} 2 & 3 \\ 4 & 7 \end{pmatrix}.$$

Linear Transformation Definition

Definition

Let U and V be real vector spaces, and suppose α and β are in $\mathbb{R}$. A **linear transformation** $T : U \to V$ satisfies

$$T(\alpha \boldsymbol{u}_1 + \beta \boldsymbol{u}_2) = \alpha T(\boldsymbol{u}_1) + \beta T(\boldsymbol{u}_2)$$

for all $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$ in U. The vector space U is the **domain** of T and V is the **codomain** of T. The set $\mathrm{Im}(T) = \{T(\boldsymbol{u}) | \boldsymbol{u} \in U\}$ is the **image** or **range** of T.

Linear Transformation Example

Example

Prove $F : \mathbb{R}^3 \to \mathbb{R}^2$ defined by $F : (x, y, z) \mapsto (x, y)$ is a linear transformation.

Kernel Definition

Definition

The **kernel** of a linear transformation $T : U \to V$ is

$$\mathrm{Ker}(T) = \{\boldsymbol{u} | T(\boldsymbol{u}) = \mathbf{0}\}.$$

Kernel Example

Example

Define $T : \mathbb{R}^3 \to \mathbb{R}^2$ via

$$T\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} -1 & 0 & 1 \\ 5 & 1 & -5 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

Find $\mathrm{Ker}(T)$.

Finding the Kernel Using Python

You can use the function `scipy.linalg.null_space` to find the kernel.

See the SciPy documentation for more details.

Kernel Theorems

Theorem

Suppose $T : U \to V$. The set $Ker(T)$ is a subspace of U.

Theorem (Rank-Nullity Theorem)

Let U be a vector space. Let $T : U \to V$ be a linear transformation of U into another vector space V. Then

$$dim(U) = \overbrace{dim\ Im(T)}^{rank} + \underbrace{dim\ Ker(T)}_{nullity}$$

Coordinate Representation of a Vector

For V a vector space with basis $\mathcal{B} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n)$. We can represent an arbitrary $\boldsymbol{w}$ in V using the unique linear combination of the elements of $\mathcal{B}$. Specifically,

$$\boldsymbol{w} = \alpha_1 \boldsymbol{v}_1 + \alpha_2 \boldsymbol{v}_2 + \ldots + \alpha_n \boldsymbol{v}_n.$$

Using this, we can write the coordinate representation of $\boldsymbol{w}$ with respect to the basis $\mathcal{B}$:

$$(\boldsymbol{w})_{\mathcal{B}} = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{pmatrix}.$$

Linear Transformations and Matrices

There is a matrix representation of any linear transformation between finite dimensional vector spaces. Consider vector space U with ordered basis $\mathcal{B} = (\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_m)$, and vector space V with ordered basis $\mathcal{C} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n)$. Suppose $T : U \to V$ such that

$$T(u_j) = \alpha_{1j}\boldsymbol{v}_1 + \alpha_{2j}\boldsymbol{v}_2 + \ldots + \alpha_{nj}\boldsymbol{v}_n.$$

Then the matrix representation in terms of these two bases is

$$\mathcal{M}(T)_{\mathcal{B}}^{\mathcal{C}} = \begin{pmatrix} \alpha_{11} & \ldots & \alpha_{1m} \\ \vdots & \ddots & \vdots \\ \alpha_{n1} & \ldots & \alpha_{nm} \end{pmatrix}.$$

Linear Transformations and Matrices Example

Example

Consider $F : \mathbb{R}^3 \to \mathbb{R}^2$ defined by $F(x, y, z) = (x, y)$. Write the matrix representation of F in terms of the standard basis.

Change of Basis

Suppose we have bases $\mathcal{B} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n)$ and $\mathcal{C} = (\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_n)$ for a vector space V. Let $T : V \to V$. Then

$$\mathcal{M}(T)_{\mathcal{C}}^{\mathcal{C}} = N^{-1}\mathcal{M}(T)_{\mathcal{B}}^{\mathcal{B}}N,$$

where N is the $n \times n$ matrix whose columns are the basis elements of $\mathcal{C}$ written in terms of the basis $\mathcal{B}$. That is,

$$N = \Big((\boldsymbol{w}_1)_{\mathcal{B}} \ (\boldsymbol{w}_2)_{\mathcal{B}} \ \ldots (\boldsymbol{w}_n)_{\mathcal{B}} \Big).$$

Change of Basis Example

Example

Consider

$$\mathcal{M}(T) = \begin{pmatrix} 2 & -4 \\ 6 & 2 \end{pmatrix}.$$

Write the matrix representation of T in terms of the basis

$$\mathcal{B} = \left(\begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right).$$

Python Code

```python
import numpy as np
from scipy.linalg import null_space

# Change of basis function
def change_matrix_basis(matrix, new_basis):
    """
    matrix: nxn matrix written in original basis
    new_basis: nxn matrix where column j represents
        the j-th basis element written in terms of
        the original basis

    return: matrix written in terms of the new basis
    """

    # Check to verify that new_basis is actually a basis
    if null_space(new_basis).shape[1] != 0:

        raise Exception('This is not a basis!')

    # Calculate matrix written in new basis
    new_matrix = np.linalg.inv(new_basis) @ matrix @ new_basis

    # Round since float accuracy makes numbers slightly off
    new_matrix = np.round(new_matrix, 6)

    return new_matrix
```

Inner Product Definition

Definition

Let V be a real vector space, and suppose $\boldsymbol{u}$, $\boldsymbol{v}$, and $\boldsymbol{w}$ are arbitrary elements of V. An **inner product** on V is a bivariate function $\langle \cdot, \cdot \rangle : V \times V \to \mathbb{R}$ with the following properties.

IP1 The inner product $\langle \boldsymbol{v}, \boldsymbol{v} \rangle \geq 0$ and $\langle \boldsymbol{v}, \boldsymbol{v} \rangle = 0$ if and only if $\boldsymbol{v} = \mathbf{0}$.

IP2 $\langle \boldsymbol{v}, \boldsymbol{w} \rangle = \langle \boldsymbol{w}, \boldsymbol{v} \rangle$.

IP3 For all α and β in $\mathbb{R}$, $\langle \alpha \boldsymbol{u} + \beta \boldsymbol{v}, \boldsymbol{w} \rangle = \alpha \langle \boldsymbol{u}, \boldsymbol{w} \rangle + \beta \langle \boldsymbol{v}, \boldsymbol{w} \rangle$.

The Dot Product

The most common inner product example is the "dot product" in $\mathbb{R}^n$.

Suppose

$$\boldsymbol{u} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{pmatrix} \qquad \text{and} \qquad \boldsymbol{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}.$$

Then this inner product is

$$\langle \boldsymbol{u}, \boldsymbol{v} \rangle = \boldsymbol{u} \bullet \boldsymbol{v} = \boldsymbol{u}^T \boldsymbol{v} = u_1 v_1 + u_2 v_2 + \ldots + u_n v_n.$$

In Python, the command is `np.dot`, though you can also do `u.T @ v` provided that the dimensions are properly defined.

Another Inner Product Example

Consider the vector space of continuous functions on the interval $[a, b]$.

Then an inner product is

$$\langle f, g \rangle = \int_a^b f(x)g(x)\ dx.$$

Norm Definition

Definition

Suppose V is a a real vector space, $\boldsymbol{u}$ and $\boldsymbol{v}$ are in V, and α is in $\mathbb{R}$. A **norm** is a real valued function $\|\cdot\| : V \to \mathbb{R}$ with the following properties.

N1 $\|\boldsymbol{v}\| \geq 0$ and $\|\boldsymbol{v}\| = 0$ if and only if $\boldsymbol{v} = \mathbf{0}$.

N2 $\|\alpha\boldsymbol{v}\| = |\alpha|\|\boldsymbol{v}\|$.

N3 $\|\boldsymbol{u} + \boldsymbol{v}\| \leq \|\boldsymbol{u}\| + \|\boldsymbol{v}\|$.

Norm Examples

For the real vector space $\mathbb{R}^n$, the Euclidean norm is the most common. For $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$, it is defined to be

$$\|\boldsymbol{x}\| = \sqrt{\boldsymbol{x} \bullet \boldsymbol{x}} = \sqrt{x_1^2 + x_2^2 + \ldots + x_n^2}.$$

In Python, the command is `np.linalg.norm`.

Other Norm Examples

- For the real vector space $\mathbb{R}^n$, one example is the ℓ_p-norm where $p \geq 1$. It is defined as

$$\|\boldsymbol{x}\|_p = \left(|x_1|^p + |x_2|^p + \ldots + |x_n|^p\right)^{1/p}.$$

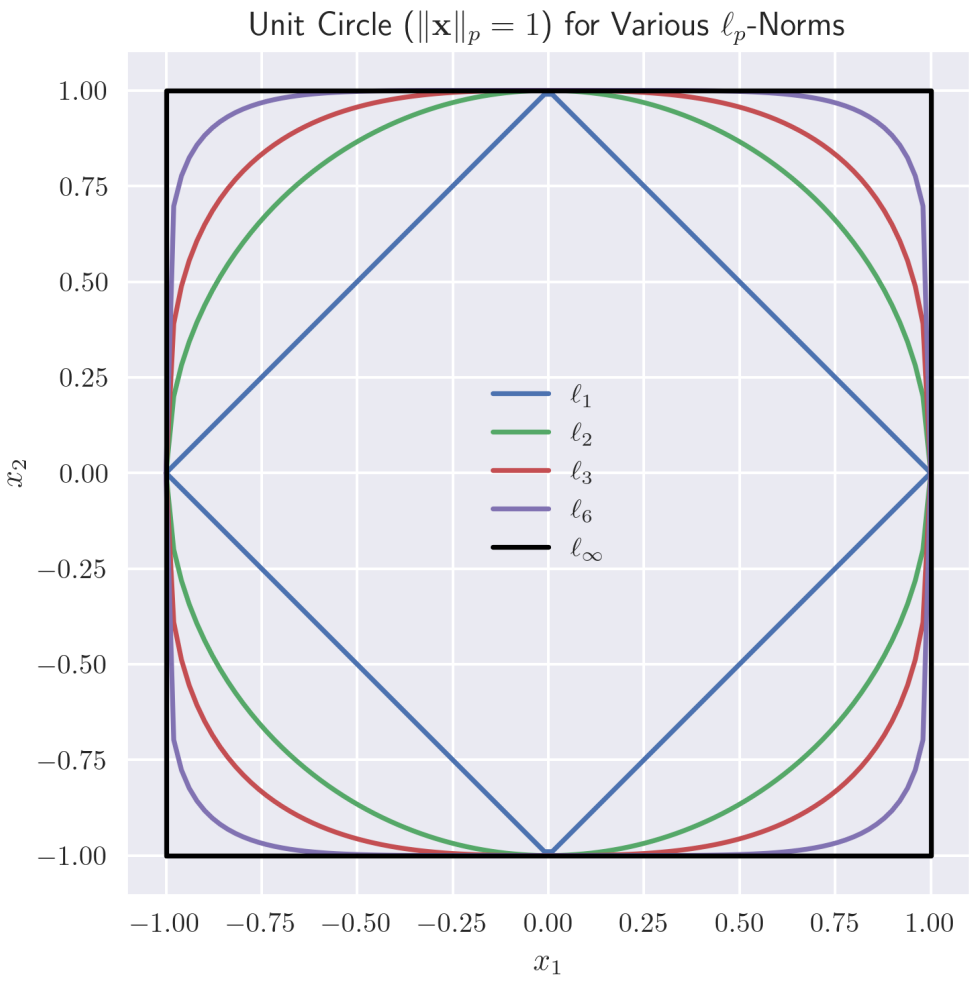
- The limiting case of the ℓ_p-norm is the ℓ_∞-norm. It is defined as

$$\|\boldsymbol{x}\|_\infty = \max_i |x_i|.$$

- For the vector space of continuous real-valued functions on $[a, b]$, if $p \geq 1$, we can define the L^p norm of f to be

$$\|f\|_p = \left(\int_a^b |f(x)|^p \, dx\right)^{1/p}.$$

Unit Circle in $\mathbb{R}^2$ for Various ℓ_p-Norms



Inner Products Induce Norms

If V is an inner product space, the norm induced by the inner product is

$$\|\boldsymbol{v}\| = \sqrt{\langle \boldsymbol{v}, \boldsymbol{v} \rangle}.$$

Law of Cosines and Inner Products

People like to think of inner products defining the angle θ between vectors

$$\langle \boldsymbol{u}, \boldsymbol{v} \rangle = \|\boldsymbol{u}\| \|\boldsymbol{v}\| \cos\theta.$$

In particular, we say $\boldsymbol{u}$ and $\boldsymbol{v}$ are **orthogonal** if

$$\langle \boldsymbol{u}, \boldsymbol{v} \rangle = 0.$$

Distance Metric Definition

Definition

A bivariate function d on a set V is a **distance metric** if for all $\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w}$ in V the following hold.

D1 $d(\boldsymbol{u}, \boldsymbol{v}) \geq 0$ and $d(\boldsymbol{u}, \boldsymbol{v}) = 0$ if and only if $\boldsymbol{u} = \boldsymbol{v}$

D2 $d(\boldsymbol{u}, \boldsymbol{v}) = d(\boldsymbol{v}, \boldsymbol{u})$

D3 $d(\boldsymbol{u}, \boldsymbol{v}) + d(\boldsymbol{v}, \boldsymbol{w}) \geq d(\boldsymbol{u}, \boldsymbol{w})$

Distance Metric Examples

- For the real vector space $\mathbb{R}^n$, the Euclidean distance is most common. Given $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$ and $\boldsymbol{y} = (y_1, y_2, \ldots, y_n)$, it is defined to be

$$d(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \ldots + (x_n - y_n)^2}.$$

- The discrete metric on any set:

$$d(\boldsymbol{x}, \boldsymbol{y}) = \begin{cases} 1 & \boldsymbol{x} \neq \boldsymbol{y} \\ 0 & \boldsymbol{x} = \boldsymbol{y}. \end{cases}$$

- On the set of continuous real-valued functions on the interval [a, b], we can define

$$d(f, g) = \sqrt{\int_a^b \Big(f(x) - g(x)\Big)^2 \, dx}.$$

Norms Induce Distances

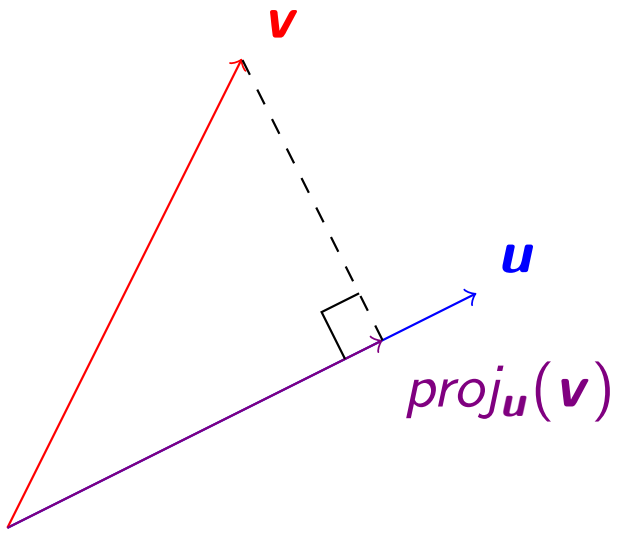
The distance metric induced by a norm is

$$d(\boldsymbol{u}, \boldsymbol{v}) = \|\boldsymbol{u} - \boldsymbol{v}\|.$$

Projection

The projection of $\boldsymbol{v}$ onto $\boldsymbol{u}$ is

$$proj_{\boldsymbol{u}}(\boldsymbol{v}) = \frac{\langle \boldsymbol{u}, \boldsymbol{v} \rangle}{\|\boldsymbol{u}\|^2} \boldsymbol{u}.$$



Projection Example

Example

Suppose

$$\boldsymbol{u} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \qquad \text{and} \qquad \boldsymbol{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Compute $proj_{\boldsymbol{u}}(\boldsymbol{v})$.

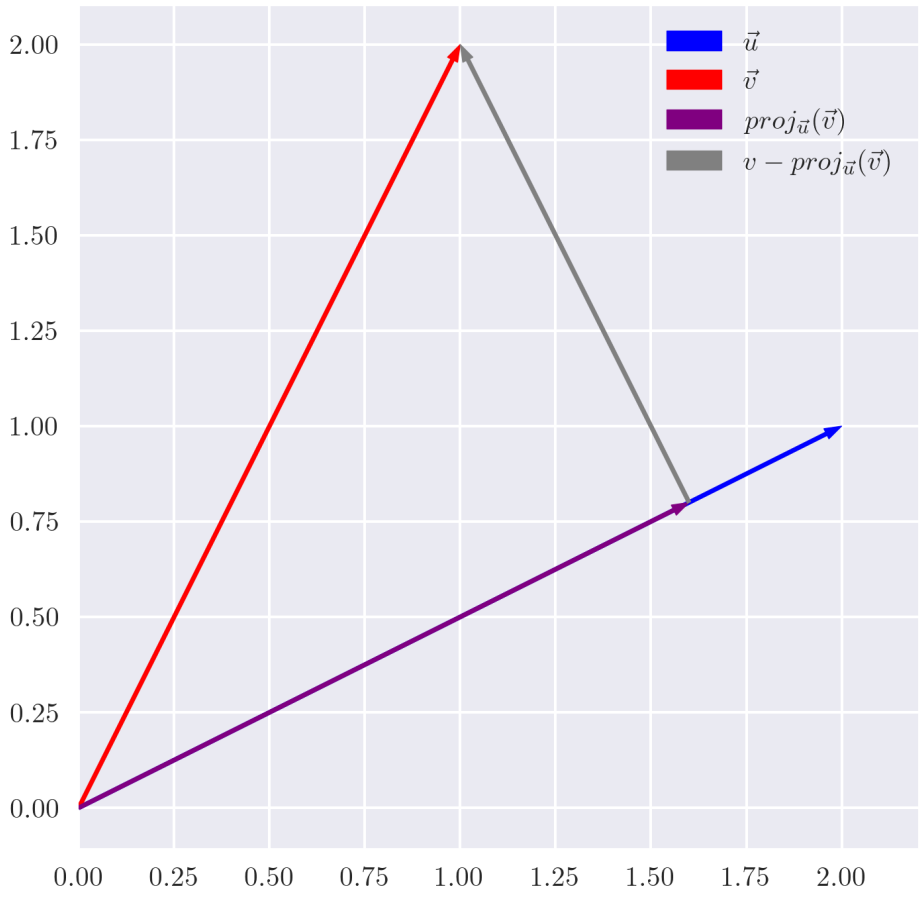
Projection Solution Python Code

```python
# Import modules
import numpy as np
import matplotlib.pyplot as plt

# Use LaTex and increase image resolution
plt.rcParams['text.usetex'] = True
plt.rcParams['figure.dpi'] = 300

# Use Seaborn style
plt.style.use('seaborn-v0_8')

# Define u and v
u, v = np.array([2, 1]), np.array([1, 2])

# Calculate projection; key step
proj = np.dot(u, v)/np.linalg.norm(u)**2 * u

# Calculate the part of v perpendicular to u
u_perp = v - proj

# Define origin
origin = np.array([0, 0])
```

Projection Solution Python Code

```
# Plot figure
fig, ax = plt.subplots(1, 1)

# Draw arrow for u
ax.arrow(*origin, *u, label = r'$\vec{u}$', color = 'blue',
  width = 0.01, length_includes_head = True)

# Draw arrow for v
ax.arrow(*origin, *v, label = r'$\vec{v}$', color = 'red',
  width = 0.01, length_includes_head = True)

# Draw arrow for projection
ax.arrow(*origin, *proj, label = r'$proj_{\vec{u}}(\vec{v})$',
         color = 'purple', width = 0.01,
         length_includes_head = True)

# Draw arrow for part of v perpendicular to u
# Place initial side at terminal side of projection
ax.arrow(*proj, *u_perp, label = r'$v - proj_{\vec{u}}(\vec{v})$',
         color = 'gray', width = 0.01,
         length_includes_head = True)

# Add a little horizontal space for legend
ax.set_xlim([0, np.max([u[0], v[0], proj[0], u_perp[0]]) + 0.2])

# Make aspect ratio equal
fig.gca().set_aspect('equal')

# Place legend at upper right
ax.legend(loc = 'upper right')

# Save the figure
plt.savefig(path + r'ex2-1.png')

# Show graph
plt.show()
```

Projection Result



Gram-Schmidt Orthogonalization Process

Theorem

If $(\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n)$ *is a sequence of linearly independent vectors in an inner product space* V*, then the sequence* $(\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n)$*, defined by*

$$\boldsymbol{u}_1 = \boldsymbol{v}_1 \qquad \text{and} \qquad \boldsymbol{u}_k = \boldsymbol{v}_k - \sum_{i=1}^{k-1} \frac{\langle \boldsymbol{v}_k, \boldsymbol{u}_i \rangle}{\|\boldsymbol{u}_i\|^2} \boldsymbol{u}_i$$

for $k = 2, 3, \ldots, n$*, is an orthogonal sequence in* V *with the property that*

$$span\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n\} = span\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}.$$

Gram-Schmidt Orthogonalization Process Example

Example

Orthogonalize the first two vectors of the basis $(1, x, x^2, \ldots)$ for the set of polynomials over $[0, 1]$ with inner product

$$\langle p, q \rangle = \int_0^1 p(x)q(x)\ dx.$$

Orthogonal Basis

If $(\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n)$ is an orthogonal basis of V, then for $\boldsymbol{v}$ in V, we have

$$\boldsymbol{v} = \alpha_1 \boldsymbol{u}_1 + \alpha_2 \boldsymbol{u}_2 + \ldots + \alpha_n \boldsymbol{u}_n$$

implies

$$\alpha_i = \frac{\langle \boldsymbol{u}_i, \boldsymbol{v} \rangle}{\|\boldsymbol{u}_i\|^2}.$$

Cauchy-Schwarz Inequality

Theorem (Cauchy-Schwarz Inequality)

Suppose $\boldsymbol{u}$ *and* $\boldsymbol{v}$ *are in the inner product space* V*. Then*

$$|\langle \boldsymbol{u}, \boldsymbol{v} \rangle| \leq \|\boldsymbol{u}\| \|\boldsymbol{v}\|.$$

The Projection Theorem

Definition

The **orthogonal complement** of a set $X \subseteq V$ is the set

$$X^{\perp} = \{\boldsymbol{v} \in V | \langle \boldsymbol{x}, \boldsymbol{v} \rangle = 0 \text{ for all } \boldsymbol{x} \in X\}.$$

Theorem (Projection Theorem)

If U is a finite-dimensional subspace of an inner product space V, then for each element $\boldsymbol{v}$ in V, there exists unique elements $\boldsymbol{u}$ in U and $\boldsymbol{w}$ in $U^{\perp}$ such that $\boldsymbol{v} = \boldsymbol{u} + \boldsymbol{w}$.

Best Approximation Theorem

Theorem (Best Approximation)

Consider a vector space V with inner product $\langle \cdot, \cdot \rangle$. Let U be a finite subspace of V. With respect to the norm induced by $\langle \cdot, \cdot \rangle$, the best approximation of vector $\boldsymbol{v}$ in V contained in U is

$$proj_U(\boldsymbol{v}).$$

Easy Projection Example

Example

Consider the subspace U of $\mathbb{R}^3$ spanned by the orthogonal vectors

$$\boldsymbol{u}_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \qquad \text{and} \qquad \boldsymbol{u}_2 = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix}.$$

Compute the best approximation of $\boldsymbol{v} = (1, 2, 3)^T$ contained in U.

Easy Projection Python Code

```python
import numpy as np

# Define u1 and u2
u1, u2 = np.array([1, 1, 0]), np.array([1, -1, 0])

# Define v
v = np.array([1, 2, 3])

# Calculate projection

# First, projection onto u1
proj = np.dot(u1, v)/np.linalg.norm(u1)**2 * u1

# Second, add projection onto u2
proj += np.dot(u2, v)/np.linalg.norm(u2)**2 * u2

proj
```

The result is [1, 2, 0] as expected.

Projection Check

Example

Consider the subspace U of $\mathbb{R}^3$ spanned by the orthogonal vectors

$$\boldsymbol{u}_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \qquad \text{and} \qquad \boldsymbol{u}_2 = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix},$$

and $\boldsymbol{v} = (1, 2, 3)^T$. Randomly generate elements $\boldsymbol{u}'$ in U to verify that $\|\boldsymbol{v} - proj_U(\boldsymbol{v})\| \leq \|\boldsymbol{v} - \boldsymbol{u}'\|$.

Projection Check Python Code

```python
# Calculate minimal norm
min_norm = np.linalg.norm(v - proj)

# Define number of trials
trials = 100_000

# Initialize object to hold results
norm_vals = np.zeros(trials)

for i in range(trials):

    # Generate weights; use guassian distribution
    alpha = np.random.normal(size = 2)

    # Calculate u'
    u_prime = alpha[0] * u1 + alpha[1] * u2

    # Calculate and record norm
    norm_vals[i] = np.linalg.norm(v - u_prime)

print(f'The number of norms less than the norm of v - proj_U(v) is {np.sum(norm_vals <
    min_norm)}.')
```

The output reads:

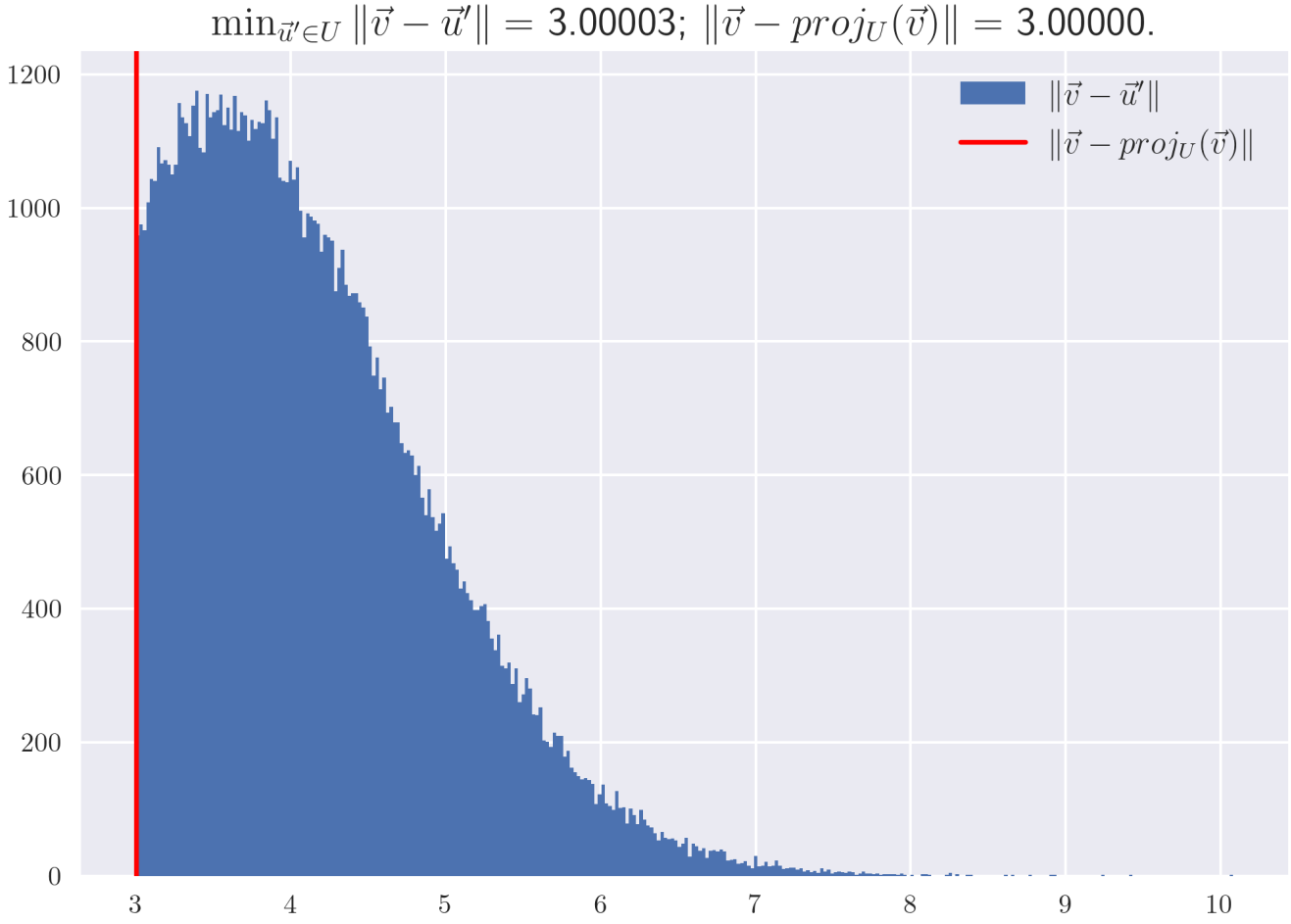
The number of norms less than the norm of v - proj_U(v) is 0.

This was the expected result!

Projection Check Histogram Python Code

```
# Use LaTex and increase image resolution
plt.rcParams['text.usetex'] = True
plt.rcParams['figure.dpi'] = 300

# Use Seaborn style
plt.style.use('seaborn-v0_8')

# Plot histogram
plt.hist(norm_vals, label = r"$\|\vec{v} - \vec{u}'\|$", bins = int(np.sqrt(trials)))

# Plot vertical line
plt.axvline(min_norm, ymin = 0, ymax = 1, label = r'$\| \vec{v} - proj_U(\vec{v})\|$',
            color = 'red')

# Give plot a title
plt.title(r"$\min_{\vec{u}'\in U} \| \vec{v} - \vec{u}'\| = $ "
      + f'{np.min(norm_vals):.5f};'
      + r' $\|\vec{v} - proj_{U}(\vec{v})\| = $ '
      + f'{min_norm:.5f}.',
    fontsize = 15)

# Create a legend
plt.legend(fontsize = 12)

# Save the figure
plt.savefig(path + r'ex2-2.png')

plt.show()
```

Projection Check Histogram



Hard Projection Example

Example

Consider the inner product space of continuous functions on $[0, 1]$, where the inner product is

$$\langle p, q \rangle = \int_0^1 p(x)q(x)\ dx.$$

Project e^x onto the subspace spanned by the orthogonal vectors 1 and $x - 1/2$. Compare the projection with the tangent line approximation at $x = 1/2$.

Hard Projection Example Solution

Solution. The projection is

$$proj(x) = a_{proj} + b_{proj}\left(x - \frac{1}{2}\right),$$

where

$$\begin{aligned} a_{proj} &= \frac{\langle 1, e^x \rangle}{\|1\|^2} \\ &= \frac{\int_0^1 1 \cdot e^x \, dx}{\int_0^1 1^2 \, dx} \\ &= e - 1 \end{aligned} \qquad \begin{aligned} b_{proj} &= \frac{\langle x - 1/2, e^x \rangle}{\|x - 1/2\|^2} \\ &= \frac{\int_0^1 (x - 1/2) e^x \, dx}{\int_0^1 (x - 1/2)^2 \, dx} \\ &= 6(3 - e). \end{aligned}$$

The tangent line approximation is

$$tan_line(x) = a_{tl} + b_{tl}\left(x - \frac{1}{2}\right),$$

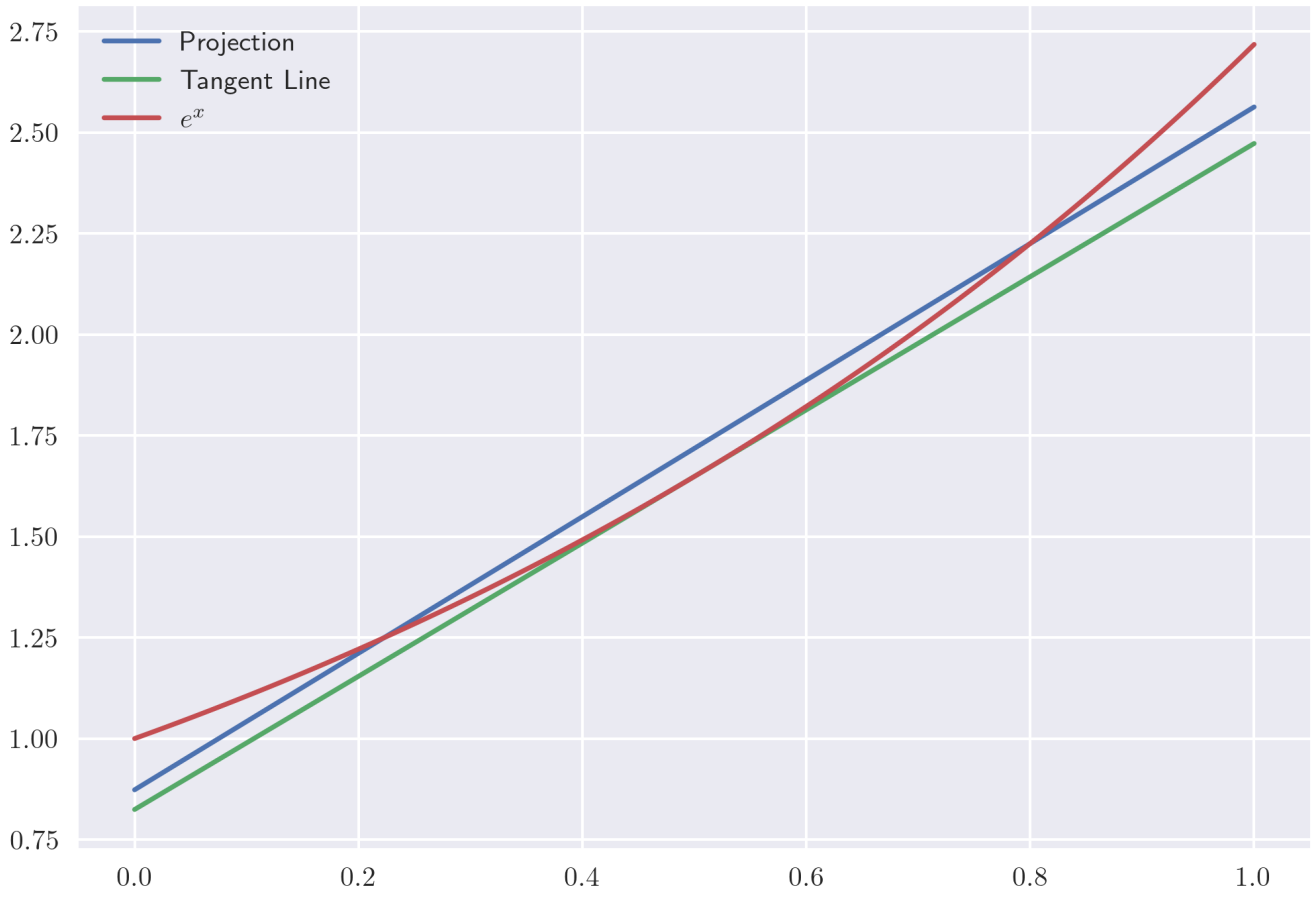
where

$$a_{tl} = e^{1/2} \qquad b_{tl} = e^{1/2}.$$

Hard Projection Example Solution Python Code

```python
# Import numerical integrator
from scipy.integrate import quad

# Define inner product
inner = lambda f, g: quad(lambda x: f(x) * g(x), 0, 1)[0]

# Define basis elements
u1, u2 = lambda x: 1, lambda x: x - 1/2

# Calculate inner products
a_proj, b_proj = inner(u1, np.exp)/inner(u1, u1), inner(u2, np.exp)/inner(u2, u2)

# Define small value
h = 1e-5

# Calculate tangent line coeffs
a_tl, b_tl = np.exp(0.5), (np.exp(0.5 + h) - np.exp(0.5 - h))/(2 * h)

# Define functions
proj = lambda x: a_proj * u1(x) + b_proj * u2(x)
tan_line = lambda x: a_tl * u1(x) + b_tl * u2(x)

# Get the x-values for plot
x_vals = np.linspace(0, 1, 100)

# Plot results
plt.plot(x_vals, proj(x_vals), label = 'Projection')
plt.plot(x_vals, tan_line(x_vals), label = 'Tangent Line')
plt.plot(x_vals, np.exp(x_vals), label = '$e^x$')

# Create a legend
plt.legend()

# Save the figure and show figure
plt.savefig(path + r'ex2-3.png')
plt.show()
```

Hard Projection Example Image



Projection Example Result

```
# Define norm
norm = lambda f: np.sqrt(inner(f, f))

# Let's calculate the norms
norm_proj = norm(lambda x: np.exp(x) - proj(x))
norm_tl = norm(lambda x: np.exp(x) - tan_line(x))

print(f'Using the projection approximation the norm is {norm_proj:.3f}.')
print(f'Using the tangent line approximation the norm is {norm_tl:.3f}.')
```

The output reads:

Using the projection approximation the norm is 0.063.

Using the tangent line approximation the norm is 0.094.

Determinant of a 2×2 Matrix

Suppose

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

Then

$$\det(A) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc.$$

Determinant Example

Example

$$\begin{vmatrix} -1 & -5 \\ 2 & 1 \end{vmatrix} =$$

Determinant of an $n \times n$ Matrix

Suppose A is the $n \times n$ matrix

$$A = (a_{ij}).$$

Let A_{ij} denote A with the i-th row and j-th column removed. Then

$$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij} \det(A_{ij})$$

for any choice of i in $\{1, 2, \ldots, n\}$.

Determinant Example

Example

$$\begin{vmatrix} 2 & 1 & 2 \\ 0 & 3 & -1 \\ 4 & 1 & 1 \end{vmatrix} =$$

Properties of Determinants

Let A and B be $n \times n$ matrices, $\boldsymbol{a}_j$, $\boldsymbol{b}$, and $\boldsymbol{c}$ be $n \times 1$ vectors, and α and β real numbers.

(a) If the columns of A are linearly dependent, $\det(A) = 0$.

(b) If A^{-1} exists, then $\det(A^{-1}) = \frac{1}{\det(A)}$.

(c) $\det(\alpha A) = \alpha^n \det(A)$.

(d) $\det(A^T) = \det(A)$

(e) $\det(AB) = \det(A)\det(B)$

(f) $\det(I) = 1$

(g) $\left|\boldsymbol{a}_1\ \boldsymbol{a}_2\ \ldots \boldsymbol{a}_{j-1}\ \alpha\boldsymbol{b} + \beta\boldsymbol{c}\ \boldsymbol{a}_{j+1} \ldots\ \boldsymbol{a}_n\right| = \alpha \left|\boldsymbol{a}_1\ \boldsymbol{a}_2\ \ldots \boldsymbol{a}_{j-1}\ \boldsymbol{b}\ \boldsymbol{a}_{j+1} \ldots \boldsymbol{a}_n\right| + \beta \left|\boldsymbol{a}_1\ \boldsymbol{a}_2\ \ldots \boldsymbol{a}_{j-1}\ \boldsymbol{c}\ \boldsymbol{a}_{j+1} \ldots \boldsymbol{a}_n\right|$.

(h) $\left|\boldsymbol{a}_1\ \boldsymbol{a}_2\ \ldots \boldsymbol{a}_j\ \boldsymbol{a}_{j+1} \ldots\ \boldsymbol{a}_n\right| = -\left|\boldsymbol{a}_1\ \boldsymbol{a}_2\ \ldots \boldsymbol{a}_{j+1}\ \boldsymbol{a}_j \ldots\ \boldsymbol{a}_n\right|$

Property (a) is very important. In `numpy`, there's `np.linalg.det` which computes the determinant. Since computers will be available to you in most circumstances the other properties are less important.

Cramer's Rule

Consider a system of n linear equations with n unknowns

$$A\boldsymbol{x} = \boldsymbol{b}$$

where A is an $n \times n$ matrix with nonzero determinant and

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}.$$

Then

$$x_i = \frac{\det(A_i)}{\det(A)},$$

where A_i is the matrix formed by replacing the i-th column of A by the column vector $\boldsymbol{b}$.

Cramer's Rule Example

Example

Solve the system

$$\begin{aligned} 3x \quad &+2y \quad -2z \quad &= 1 \\ 2x \quad &-y \quad &= 0 \\ &-4y \quad +3z \quad &= 1. \end{aligned}$$

Python Implementation of Cramer's Rule

```python
# Import modules
import numpy as np

# Define function which uses Cramer's rule to solve system
def solve_cramer(A, b):

    # Record number of columns in A
    ncols = A.shape[1]

    # Initialize array to hold solution
    x = np.zeros(ncols)

    # Calculate det(A)
    det_A = np.linalg.det(A)

    for i in range(ncols):

        # Make a copy of A
        A_i = A.copy()

        # Replace the i-th columns with b
        # This would change A if we didn't make copy
        A_i[:, i] = b

        # Calculate det(A_i)
        det_A_i = np.linalg.det(A_i)

        # Calculate and save i-th component of solution
        x[i] = det_A_i/det_A

    return x
```

Eigenvector and Eigenvalue Definitions

Definition

Let V be a vector space and consider a linear transformation with matrix representation A. An element $\boldsymbol{v}$ in V is an **eigenvector** of A if there exists a scalar λ such that $A\boldsymbol{v} = \lambda\boldsymbol{v}$. If $\boldsymbol{v} \neq \mathbf{0}$, then λ is called an **eigenvalue** of A.

In `numpy`, we have `np.linalg.eig`. The function computes the eigenvalues and eigenvectors, respectively.

Eigenvector and Eigenvalue Example

Example

Let $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. Show that $\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $\boldsymbol{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ are eigenvectors. What are their eigenvalues?

Eigenvectors and Kernels

If $\boldsymbol{v}$ is an eigenvector of A with eigenvalue λ, then

$$A\boldsymbol{v} = \lambda\boldsymbol{v} \qquad \text{implies} \qquad (A - \lambda I)\boldsymbol{v} = \mathbf{0}.$$

So, if $\boldsymbol{v} \neq \mathbf{0}$, then $\mathrm{Ker}(A - \lambda I) \neq \{\mathbf{0}\}$. This, implies $A - \lambda I$ has linearly dependent columns. Hence, $\det(A - \lambda I) = 0$.

Characteristic Polynomial

Definition

For an $n \times n$ matrix A, the **characteristic polynomial** of A is

$$p_A(\lambda) = \det(A - \lambda I).$$

We can find the eigenvalues by finding the zeros of p_A. We can then plug the eigenvalues into $(A - \lambda I)\boldsymbol{x} = \mathbf{0}$ and solve for $\boldsymbol{x}$ to find the corresponding eigenvectors.

Using the Characteristic Polynomial

Example

Find the eigenvalues and eigenvectors of $A = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$.

Diagonalizable Matrix

Definition

We say that a matrix A is **diagonalizable** if V has a basis of eigenvectors of A.

Diagonalizable Matrix Example

In our previous example, we saw the eigenvectors of

$$A = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix},$$

are

$$\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{and} \quad \boldsymbol{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

These eigenvectors are linearly independent, so they form a basis for $\mathbb{R}^2$. As a result, A is diagonalizable. In particular, we can use the basis $(\boldsymbol{v}_1, \boldsymbol{v}_2)$ for $\mathbb{R}^2$ and the change of basis formula to diagonalize A. Under the basis, $(\boldsymbol{v}_1, \boldsymbol{v}_2)$ the matrix representation of the linear transformation corresponding to A is

$$N^{-1}AN = \begin{pmatrix} 1.5 & 0 \\ 0 & 0.5 \end{pmatrix},$$

where N is the matrix which contains the basis elements $(\boldsymbol{v}_1, \boldsymbol{v}_2)$ as columns, i.e.

$$N = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Recall: $\boldsymbol{v}_1$ has eigenvalue 1.5 and $\boldsymbol{v}_2$ has eigenvalue 0.5.

Diagonalizable Matrix Example

Example

Suppose $A = \begin{pmatrix} 1 & 2 \\ 3 & 2 \end{pmatrix}$. Find a basis of eigenvectors as well as their eigenvalues. Use the result to diagonalize A.

Eigenvector and Eigenvalue Python Example

Example

Suppose $A = \begin{pmatrix} 4 & 0 & 1 \\ -2 & 1 & 0 \\ -2 & 0 & 1 \end{pmatrix}$. Find a basis of eigenvectors as well as their eigenvalues.

Eigenvectors and Eigenvalues Python Code and Result

```python
import numpy as np

# Define matrix
A = np.array([[4, 0, 1], [-2, 1, 0], [-2, 0, 1]])

# Get the eigenvalues and eigenvectors
evals, evecs = np.linalg.eig(A)

# Loop through results
for lam, v in zip(evals, evecs.T):

    print(f'eigenvalue: {lam:.2f}; eigenvector: {v}\n')
```

The output is shown below:

```
eigenvalue: 1.00; eigenvector: [0. 1. 0.]

eigenvalue: 3.00; eigenvector: [ 0.57735027 -0.57735027 -0.57735027]

eigenvalue: 2.00; eigenvector: [-0.33333333  0.66666667  0.66666667]
```

Symmetric Matrix

Definition

Suppose V is an inner product space, and $T : V \to V$. Then T is **symmetric** if we have the relation

$$\langle T\boldsymbol{v}, \boldsymbol{w} \rangle = \langle \boldsymbol{v}, T\boldsymbol{w} \rangle$$

for all $\boldsymbol{v}$ and $\boldsymbol{w}$ in V.

For the dot product on $\mathbb{R}^n$, if T has matrix representation A then symmetric means $A = A^T$.

Spectral Theorem

Theorem (Spectral Theorem)

Let V be a finite dimensional non-trivial inner product space over the real numbers, and suppose $T : V \to V$ is a symmetric linear transformation with matrix representation A. Then V has an orthogonal basis consisting of eigenvectors of A.

Positive Definite Matrix Example

Definition

Suppose A is an $n \times n$ matrix and $\mathbf{x}$ is in $\mathbb{R}^n$.

- A is **positive definite** if and only if it is symmetric and $\boldsymbol{x}^T A\boldsymbol{x} > 0$ for all $\boldsymbol{x} \neq \mathbf{0}$.
- A is **positive semidefinite** if and only if it is symmetric and $\boldsymbol{x}^T A\boldsymbol{x} \geq 0$ for all $\boldsymbol{x}$.

Positive Definite Matrix Example

Example

Show that the 2×2 identity matrix I is positive definite.

Properties of Positive Definite Matrices

Theorem

(i) *A symmetric matrix is positive definite if all of its eigenvalues are positive.*

(ii) *If A is positive definite, then it is invertible and $\det(A) > 0$.*

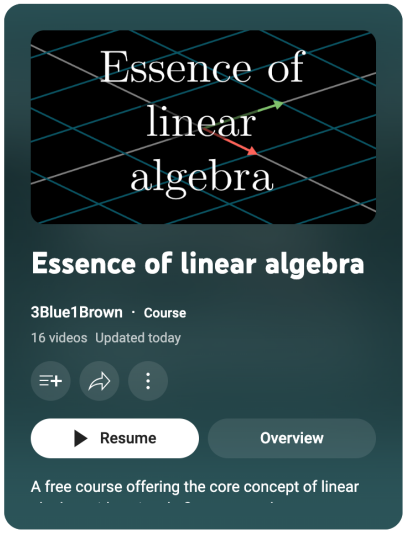
Positive Definite Matrix Example

Example

If A is positive definite, show that A^k is positive definite for all integers $k \geq 1$.

Linear Algebra on YouTube

Watch the 3Blue1Brown linear algebra video series. The series includes most of what has been covered here as well as other great material (`https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab`).



Multivariable Calculus

Partial Derivative Definition

Definition

Suppose we have $f : \mathbb{R}^n \to \mathbb{R}$. Then the **partial derivative** of f with respect to x_i is

$$\frac{\partial f}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, x_2, \ldots, x_i + h, \ldots, x_n) - f(x_1, x_2, \ldots, x_i, \ldots, x_n)}{h}.$$

The most common case is $f : \mathbb{R}^2 \to \mathbb{R}$ and $z = f(x, y)$. Then the two partials are

$$f_x(x, y) = \frac{\partial z}{\partial x} = \lim_{h \to 0} \frac{f(x + h, y) - f(x, y)}{h}$$

and

$$f_y(x, y) = \frac{\partial z}{\partial y} = \lim_{h \to 0} \frac{f(x, y + h) - f(x, y)}{h}.$$

Clairaut's Theorem

Theorem (Clairaut)

Suppose f is defined on a disk D that contains the point (a, b). If the functions f_{xy} and f_{yx} are both continuous on D, then

$$f_{xy}(a, b) = f_{yx}(a, b).$$

Difference Approximation

Theorem

Suppose f_x and f_y exist on a rectangular region R with sides parallel to the axes and containing the points (a, b) and $(a + \Delta x, b + \Delta y)$. Suppose f_x and f_y are continuous at the point (a, b), and let

$$\Delta z = f(a + \Delta x, b + \Delta y) - f(a, b).$$

Then

$$\Delta z = f_x(a, b)\Delta x + f_y(a, b)\Delta y + \varepsilon_1 \Delta x + \varepsilon_2 \Delta y$$

where $\varepsilon_1, \varepsilon_2 \to 0$ as $\Delta x, \Delta y \to 0$.

Difference Approximation Example

Example

Suppose $f(x, y, z) = \sqrt{xyz}$. Approximate $f(3.9, 4.2, 3.8)$.

The Chain Rule

Suppose z is a differentiable function of n variables $x_1, x_2, \ldots, x_n$ and each x_j is a function of the m variables $t_1, t_2, \ldots, t_m$ such that the partial derivative $\frac{\partial x_j}{\partial t_i}$ exists each for all i and j. Then z is a function of $t_1, t_2, \ldots, t_m$ and

$$\frac{\partial z}{\partial t_i} = \frac{\partial z}{\partial x_1}\frac{\partial x_1}{\partial t_i} + \frac{\partial z}{\partial x_2}\frac{\partial x_2}{\partial t_i} + \ldots + \frac{\partial z}{\partial x_n}\frac{\partial x_n}{\partial t_i}$$

for any i.

Chain Rule Example

Example

Consider the Black-Scholes differential equation

$$\frac{\partial V}{\partial t} + \frac{\sigma^2}{2} S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} = rV.$$

Rewrite the partial differential equation using the substitution $z = \ln S$.

Implicit Differentiation

Suppose that $F(x, y) = 0$ and $y = f(x)$. Then

$$\frac{\partial F}{\partial x} + \frac{\partial F}{\partial y}\frac{dy}{dx} = 0.$$

So,

$$\frac{dy}{dx} = -\frac{\frac{\partial F}{\partial x}}{\frac{\partial F}{\partial y}} = -\frac{F_x}{F_y}$$

as long as $\frac{\partial F}{\partial x}$ is continuous and $\frac{\partial F}{\partial y}$ is both continuous and nonzero.

Implicit Differentiation Example

Example

Find $\frac{dy}{dx}$ if $x^3 + y^3 = 6xy$.

Gradient Vector Definition

Definition

Suppose $\boldsymbol{x}$ is an element of $\mathbb{R}^n$. Define the **gradient vector** as

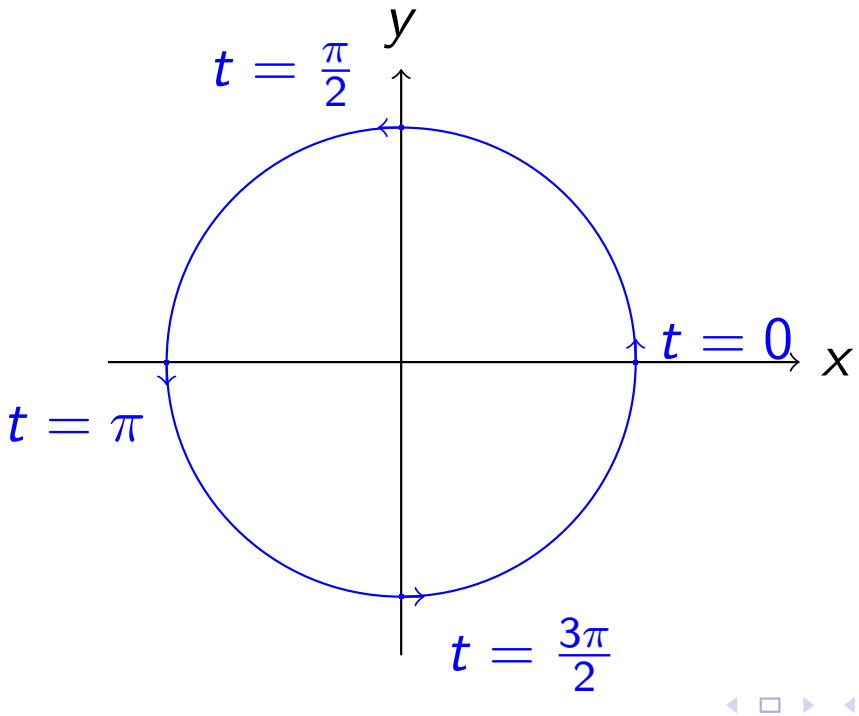
$$\nabla f(\boldsymbol{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right).$$

Space Curve

A curve in $\mathbb{R}^n$ can be defined in terms of a vector valued function $\boldsymbol{r}$, where $\boldsymbol{r} : I \to \mathbb{R}^n$ and I is some subset of $\mathbb{R}$.

For example, consider $\boldsymbol{r} : [0, 2\pi) \to \mathbb{R}^2$ such that $\boldsymbol{r} : t \mapsto (\cos t, \sin t)$.

This defines a circle oriented counterclockwise in $\mathbb{R}^2$.

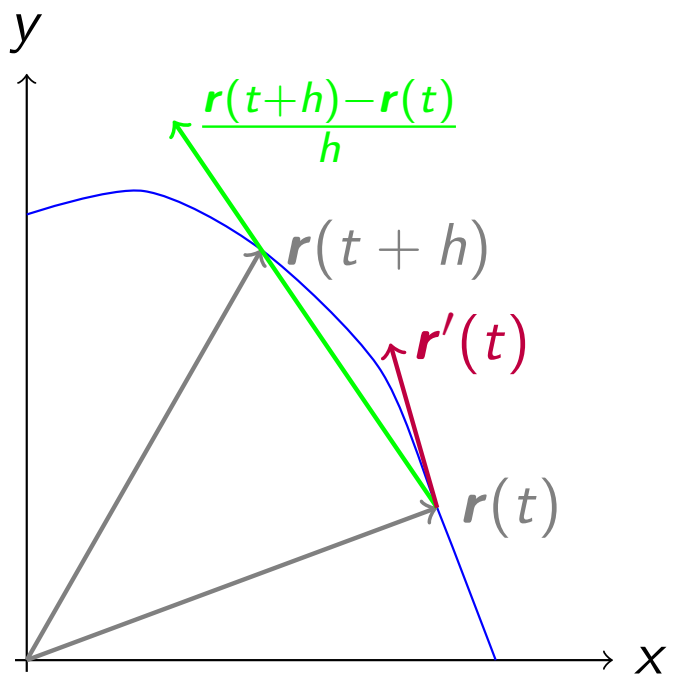


Derivative of a Space Curve

The derivative of $\boldsymbol{r} : I \to \mathbb{R}^n$ is defined to be

$$\boldsymbol{r}'(t) = \lim_{h \to 0} \frac{\boldsymbol{r}(t+h) - \boldsymbol{r}(t)}{h}.$$

The derivative is tangent to the curve described by $\boldsymbol{r}$.



Gradient Vector Orthogonal to Surface

Suppose that we have a surface defined by $F(\boldsymbol{x}) = k$ where $\boldsymbol{x}$ is in $\mathbb{R}^n$. Consider any curve defined by $\boldsymbol{r} : I \to \mathbb{R}^n$. Then we have the value of the function at $\boldsymbol{r}(t)$ is

$$F(\boldsymbol{r}(t)) = k.$$

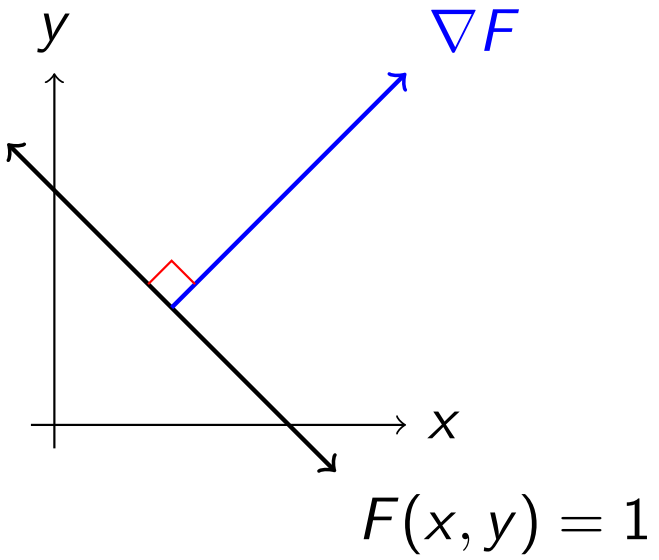
The chain rule implies

$$\nabla F(\boldsymbol{r}(t)) \bullet \boldsymbol{r}'(t) = 0.$$

The vector $\boldsymbol{r}'(t)$ is parallel to the surface. Hence, $\nabla F(\boldsymbol{r}(t))$ must be orthogonal. Since $\boldsymbol{r}$ was arbitrary, $\nabla F(\boldsymbol{x})$ must be orthogonal to $F(\boldsymbol{x}) = k$.

Gradient Vector Example

Consider $F(x, y) = x + y = 1$. Then $\nabla F = (1, 1)$. We can see this vector is orthogonal to our line (or "surface" to use the more general term).



Local Maximum and Minimum Definition

Definition

- A function of two variables has a **local maximum** at (a, b) if $f(x, y) \leq f(a, b)$ for all points (x, y) in some disk with center (a, b). The number $f(a, b)$ is called a **local maximum value**.
- A function of two variables has a **local minimum** at (a, b) if $f(x, y) \geq f(a, b)$ for all points (x, y) in some disk with center (a, b). The number $f(a, b)$ is called a **local minimum value**.

Note: A disk of radius r and center (h, k) is the set

$$\{(x, y) \in \mathbb{R}^2 | \ (x - h)^2 + (y - k)^2 < r\}.$$

Local Maximum and Minimum

Theorem

If f has a local extremum at (a, b) and the first-order partial derivatives of f exist there, then $f_x(a, b) = f_y(a, b) = 0$.

Second Derivatives Test

Suppose the second partial derivatives of f are continuous in a disk with center (a, b), and suppose that $f_x(a, b) = f_y(a, b) = 0$. Let

$$H = \begin{pmatrix} f_{xx}(a, b) & f_{xy}(a, b) \\ f_{yx}(a, b) & f_{yy}(a, b) \end{pmatrix}.$$

(a) If $\det(H) > 0$ and $f_{xx}(a, b) > 0$, then $f(a, b)$ is a local minimum.

(b) If $\det(H) > 0$ and $f_{xx}(a, b) < 0$, then $f(a, b)$ is a local maximum.

(c) If $\det(H) < 0$, then $f(a, b)$ is not a local extremum.

(d) If $\det(H) = 0$, then the test fails.

Second Derivatives Test Example

Example

Find the shortest distance from the point $(1, 0, -2)$ to the plane

$x + 2y + z = 4$.

Minimization Python Example

Example

Use Python to verify the previous example.

```
import numpy as np
from scipy.optimize import minimize

# Define function
def distance(pt):

    # Get the x- and y-values
    x, y = pt[0], pt[1]

    # Define z
    z = 4 - x - 2 * y

    return np.sqrt((x - 1)**2 + y**2 + (z + 2)**2)

# Minimize distance; initial guess at (0, 0)
minimize(distance, x0 = [0, 0])
```

Another option is to use the constraint x + 2y + z - 4 = 0 and optimize with three variables.

Minimization Python Result

Note that

$$\frac{11}{6} \approx 1.83, \qquad \frac{5}{3} \approx 1.67 \qquad \text{and} \qquad \frac{5\sqrt{6}}{6} \approx 2.04$$

```
     fun: 2.0412414523198583
hess_inv: array([[ 1.71207714, -0.67961111],
       [-0.67961111,  0.68098713]])
     jac: array([9.23871994e-07, 1.51991844e-06])
 message: 'Optimization terminated successfully.'
    nfev: 32
     nit: 7
    njev: 8
  status: 0
 success: True
       x: array([1.83333386, 1.66666707])
```

Generalization

Consider $f : \mathbb{R}^n \to \mathbb{R}$. Suppose the second order partial derivatives of f are continuous. Then the Hessian of f is

$$H = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}.$$

If H is positive definite at $\boldsymbol{x}$, then the graph of f is convex at $\boldsymbol{x}$. Conversely, if the graph of f is convex at $\boldsymbol{x}$, then H is positive semidefinite at $\boldsymbol{x}$. If the graph of f is locally convex, then critical points correspond to local minima.

Multivariate Taylor Expansion and Optimization

The multivariate second-order Taylor expansion around $\boldsymbol{x}$ is

$$f(\boldsymbol{x}+\boldsymbol{h}) \approx f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^T \boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^T H \boldsymbol{h}.$$

If $\boldsymbol{x}$ is a critical point, then $\nabla f(\boldsymbol{x}) = \mathbf{0}$. If the Hessian H is also positive definite at $\boldsymbol{x}$, then for any $\boldsymbol{h} \neq \mathbf{0}$:

$$f(\boldsymbol{x}+\boldsymbol{h}) \approx f(\boldsymbol{x}) + \underbrace{\frac{1}{2}\boldsymbol{h}^T H \boldsymbol{h}}_{>0}.$$

This shows that $f(\boldsymbol{x}+\boldsymbol{h}) > f(\boldsymbol{x})$ for all $\boldsymbol{h}$ sufficiently small, confirming that $\boldsymbol{x}$ is a local minimum.

Extreme Value Theorem for Functions of Two Variables

Theorem

If f is continuous on a closed and bounded set D in $\mathbb{R}^2$, then f attains a global maximum value $f(x_1, y_1)$ and a global minimum value $f(x_2, y_2)$ at some points (x_1, y_1) and (x_2, y_2) in D.

Global Optimization

Example

Find the global maximum and minimum values of $f(x, y) = 5 - 3x + 4y$ over the region bound by $y = x^2$ and the horizontal line $y = 1$.

Global Optimization in Python

There are many global optimization techniques in Python. However, they tend to be slow and not particularly effective on functions that aren't convex or concave. See the SciPy documentation for more details.

Global optimization

`basinhopping`(func, x0[, niter, T, stepsize, ...])	Find the global minimum of a function using the basin-hopping algorithm.
`brute`(func, ranges[, args, Ns, full_output, ...])	Minimize a function over a given range by brute force.
`differential_evolution`(func, bounds[, args, ...])	Finds the global minimum of a multivariate function.
`shgo`(func, bounds[, args, constraints, n, ...])	Finds the global minimum of a function using SHG optimization.
`dual_annealing`(func, bounds[, args, ...])	Find the global minimum of a function using Dual Annealing.
`direct`(func, bounds, *[, args, eps, maxfun, ...])	Finds the global minimum of a function using the DIRECT algorithm.

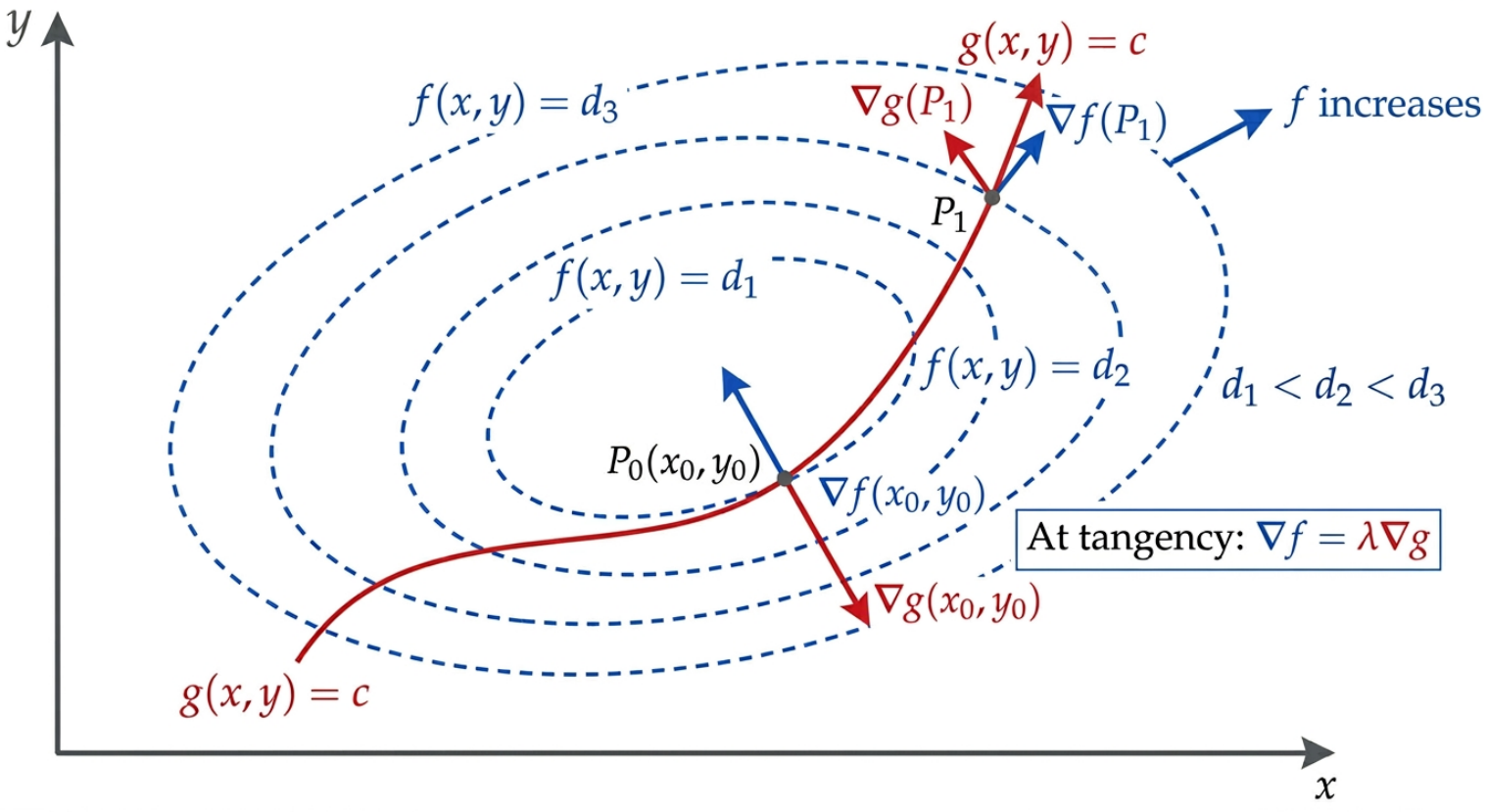
Lagrange Multipliers

For $\boldsymbol{x}$ an element of $\mathbb{R}^n$, consider $f(\boldsymbol{x})$ subject to $g(\boldsymbol{x}) = k$. If local extrema exist, they satisfy the equation

$$\nabla f(\boldsymbol{x}) = \lambda \nabla g(\boldsymbol{x}).$$

for some λ in $\mathbb{R}$.

Lagrange Multipliers Picture



Lagrange Multipliers Example

Example

Find the shortest distance from the point $(1, 0, -2)$ to the plane

$x + 2y + z = 4$.

Construction of Double the Integral

Consider $f : \mathbb{R}^2 \to \mathbb{R}$ and define the closed region

$$R = [a, b] \times [c, d] = \{(x, y) \in \mathbb{R}^2 | a \leq x \leq b,\ c \leq y \leq d\}.$$

Consider partition P of R into subrectangles $R_{ij} = [x_{i-1}, x_i] \times [y_{j-1}, y_j]$, where

$$a = x_0 \leq x_1 < \ldots < x_m = b$$

$$c = y_0 \leq y_1 < \ldots < y_n = d$$

Select tag (s_{ij}, t_{ij}) from R_{ij}. The area of R_{ij} is $\Delta A_{ij} = \Delta x_i \Delta y_j$. Define the mesh $\|P\| = \max_{i,j}\{\Delta A_{ij}\}$. Then

$$\iint_R f(x, y)\ dA = \lim_{\|P\| \to 0} \sum_{i=1}^{m} \sum_{j=1}^{n} f(s_{ij}, t_{ij})\ \Delta A_{ij}$$

whenever the limit exists.

Double Integral Example

Example

Calculate $\iint_{[0,1]\times[0,1]} xy^2 \, dA$. Use a uniform partition with n^2 subrectangles R_{ij}, and the upper right point of R_{ij} for (s_{ij}, t_{ij}).

Double Integral Python Example

Example

Use `scipy.integrate.dblquad` to verify the previous result for

$$\iint_{[0,1]\times[0,1]} xy^2 \, dA.$$

```
# See https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.dblquad.html
from scipy.integrate import dblquad

# Define f; integrate y the x
f = lambda y, x: x * y**2

# Integrate; integrate 3rd to 4th input then 2nd to 3rd input
dblquad(f, 0, 1, 0, 1)[0]
```

The output is `0.16666666666666669` which agrees with our previous result.

Fubini's Theorem

Theorem (Fubini)

Consider $R = [a, b] \times [c, d]$ *and define*

$$A_1(x) = \int_c^d f(x, y)\ dy \quad \textit{and} \quad A_2(y) = \int_a^b f(x, y)\ dx.$$

Then

$$\iint_R f(x, y)\ dA = \int_a^b A_1(x)\ dx = \int_c^d A_2(y)\ dy.$$

Fubini's Theorem Example

Example

Evaluate $\iint_R y\sin(xy)\ dA$, where $R = [1,2] \times \left[0, \frac{\pi}{2}\right]$.

Integral over General Region

Theoretically speaking, to calculate

$$\iint_D f(x,y)\ dA$$

where D is not a rectangle, simply choose rectangle R which contains D and define

$$F(x,y) = \begin{cases} f(x,y), & (x,y) \in D \\ 0, & (x,y) \in R \setminus D. \end{cases}$$

Then integrate F over R. The next slide will help show how to handle these integrals in practice.

Integral of General Region

Example

Compute $\iint_D xy^2 \, dA$, where D is the triangular region with vertices $(0,0)$, $(0,2)$, and $(1,0)$.

Jacobian Definition

Definition

The **Jacobian** of the transformation given by $x = g(u, v)$ and $y = h(u, v)$ is

$$J = \begin{pmatrix} \frac{\partial x}{\partial u} & \frac{\partial x}{\partial v} \\ \frac{\partial y}{\partial u} & \frac{\partial y}{\partial v} \end{pmatrix}.$$

Change of Variables

Suppose that we have a one-to-one transformation with continuous partial derivatives that maps the uv-plane to the xy-plane, and in particular the region S to D. Then

$$\iint_D f(x,y)\ dxdy = \iint_S f\left(x(u,v), y(u,v)\right) |\det(J)|\ dudv.$$

Change of Variables Example

Example

$$\int_{-\infty}^{\infty} e^{-x^2/2}\, dx =$$

Jacobian on YouTube

Watch the Mathemaniac video about the Jacobian on YouTube (`https://youtu.be/wCZ1VEmVjVo`).